

RLHF Pipeline Implementation

TRL: RewardTrainer → PPOTrainer → DPOTrainer — observe reward hacking in a controlled setting

30 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

AI-RL-IN-CD-000001

ID

Lab 7.5 Roadmap

1

Stage 1: Load SFT

Start from the LoRA fine-tuned model (Lab 7.2). This is π_{SFT} .

2

Stage 2: Train RM

Build pairwise preference dataset. Train RewardTrainer. Inspect reward distributions.

3

Stage 3: PPO

PPOTrainer with actor= π_{SFT} + RM. Log rewards, KL, policy loss. Observe reward hacking.

4

DPO Alternative

DPOTrainer on same preference data. Compare output quality vs PPO. Faster + simpler.

5

Compare All 3

Evaluate SFT, PPO, DPO on held-out prompts. Human evaluation style comparison.

Stage 2: Training the Reward Model

```
from trl import RewardTrainer, RewardConfig
from transformers import AutoModelForSequenceClassification

# Load SFT model with a scalar regression head
reward_model = AutoModelForSequenceClassification.from_pretrained(
    "./lora_adapter/", num_labels=1
)

# Preference dataset: each example has chosen + rejected response
# {"input_ids_chosen": [...], "input_ids_rejected": [...]}
# The RM learns: score(chosen) > score(rejected)

reward_config = RewardConfig(
    output_dir="./reward_model",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    learning_rate=1e-5,
    bf16=True,
)
reward_trainer = RewardTrainer(
    model=reward_model,
    args=reward_config,
    train_dataset=preference_dataset,
    tokenizer=tokenizer,
)
reward_trainer.train()
```

The reward model outputs a scalar for each response. Higher score = more preferred. The training loss minimises cross-entropy over pairwise comparisons.

Stage 3: DPO Training (Simpler Alternative)

```
from trl import DPOTrainer, DPOConfig
from peft import LoraConfig

# DPO needs: policy model + reference model + preference data
# Reference = frozen copy of SFT model (auto-created by TRL)
dpo_config = DPOConfig(
    output_dir="./dpo_model",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    learning_rate=5e-6,
    beta=0.1,                # KL coefficient  $\beta$ 
    bf16=True,
)
# Preference dataset: {"prompt": "...", "chosen": "...", "rejected": "..."}
dpo_trainer = DPOTrainer(
    model=sft_model,          # the policy to train
    ref_model=None,           # None = auto-create frozen copy of model
    args=dpo_config,
    train_dataset=preference_dataset,
    tokenizer=tokenizer,
    peft_config=LoraConfig(r=16, lora_alpha=32,
        target_modules=["q_proj", "v_proj"],
        task_type="CAUSAL_LM"),
)
dpo_trainer.train()
# DPO: no RM, no PPO, no critic – just SFT + reference + preference pairs
```

DPO with `ref_model=None` automatically creates a frozen copy of the policy as the reference. This is the recommended setup in TRL.

KEY REFERENCES

- TRL PPOTrainer. huggingface.co/docs/trl/ppo_trainer
- TRL DPOTrainer. huggingface.co/docs/trl/dpo_trainer
- TRL RewardTrainer. huggingface.co/docs/trl/reward_trainer

 **Next: Concept 25: Data Collection Strategies (AI-FT-IN-TH-000015)**